

Chapter 6

Paradigmes de programmation

6.1 Algorithme glouton

Un algorithme glouton est un algorithme qui suit le principe de faire, étape par étape, un choix localement optimal, dans l'espoir d'obtenir un résultat globalement optimal. Bien entendu, suivant les situations rencontrées l'algorithme glouton ne donnera pas nécessairement une solution optimale.

6.1.1 Exemple du problème de rendu de monnaie

Problème à résoudre

On se donne un système $\mathcal{S} = (c_0, \dots, c_{N-1})$ de monnaie. C'est-à-dire N valeurs de pièces de monnaie (ou de billets)

$c_0 < c_1 < \dots < c_N$. Par exemple, le système **euro** correspond au système euro (1, 2, 5, 10, 20, 50, 100, 200). Étant donnée une valeur $v \in \mathbb{N}^*$ (monnaie à rendre), on cherche à déterminer le nombre minimal de pièces à utiliser pour obtenir la somme v . C'est-à-dire que l'on cherche à calculer

$$\min \left\{ k \in \mathbb{N}^* \mid \exists (\alpha_1, \dots, \alpha_k) \in \mathcal{S}^k, \sum_{i=1}^k \alpha_i = v \right\}.$$

Remarquons que si l'on a $c_0 \neq 1$, alors des valeurs v (à commencer par $v = 1$) ne pourront pas être obtenues, on supposera donc dans la suite que $c_0 = 1$.

Résolution par Algorithme glouton

La méthode gloutonne consiste à choisir en premier la plus grande pièce possible, notons p sa valeur, il nous reste alors à résoudre le problème pour la nouvelle valeur $v - p$ et l'on procède ainsi, jusqu'à obtenir une somme nulle.

Exercice 6.1

- 1) Écrire la fonction
`glouton (v: int) (s: int array): int list` qui étant donnée la valeur v et le tableau s représentant le système de monnaie, renvoie la liste des pièces à utiliser, obtenue par algorithme glouton.
 - 2) Que donne l'algorithme glouton pour le système $S_{ex} = (1, 9, 30)$ et la valeur $v = 36$?
 - 3) Que donne l'algorithme glouton pour l'ancien système monétaire anglais. (1 (farthing), 4 (penny), 12 (3 pence), 16 (4 pence), 48 (shilling), 96 (florin), 120 (2-couronne), 240 (5-couronne), 480 (souverain), 960 (livre). On regardera en particulier le cas de la valeur $v = 16$.
-

L'algorithme glouton ne donne donc pas toujours la solution optimale. Un système de monnaie dans lequel l'algorithme glouton donne la solution optimale est appelé **système canonique**. On peut prouver que le système **euro** est canonique.

6.1.2 Fabrication d'un arbre couvrant minimal**Problème à résoudre**

On se donne un graphe G valué non orienté. C'est-à-dire que G est un graphe dont les arêtes sont de la forme (i, j, p) où i et j sont deux sommets tels que $i < j$ et $p \in \mathbb{R}_+$. (On dit que p est le poids de l'arête de i à j). Un arbre couvrant G' de G est un sous-graphe de G , dont les sommets sont tous ceux de G , les arêtes de G' sont quelques unes des arêtes de G et tel que G' est connexe. Il est clair que l'existence d'un arbre couvrant est subordonnée à la connexité du graphe G . On supposera donc que G est connexe dans la suite.

On appelle poids de G' , la quantité notée $w(G')$ égale à la somme des poids de toutes les arêtes de G' . Parmi tous les arbres couvrants de G , ceux dont la somme des poids des arêtes est minimale sont appelés **arbre couvrants minimaux**. La graphe connexe G étant donné, on cherche à construire un graphe couvrant minimal G' .

Résolution gloutonne:

L'algorithme de Kruskal consiste à fabriquer le graphe G' en partant d'un graphe de mêmes sommets que G et sans aucune arête et en ajoutant à chaque étape l'arête de plus petit poids reliant deux sommets n'appartenant pas à la même composante connexe.

Exercice 6.2

On suppose que les sommets de G sont numérotés de 0 à $n-1$, et l'on représente G informatiquement par la variable entière n et la liste l des arêtes. Une arête étant un triplet (w, i, j) , i et j codant les sommets i et j de l'arête tels que $i < j$ et w est un entier qui représente le poids de l'arête.

- 1) Programmer l'algorithme de Kruskal.
 - 2) Évaluer sa complexité.
 - 3) Prouver que l'algorithme de Kruskal renvoie une solution optimale.
-

6.1.3 problème d'allocation de salles**Problème à résoudre**

Un gestionnaire de salle reçoit un certain nombre de demandes d'occupations de sa salle. Chaque demande est caractérisée par un numéro de demande, un temps de début d'occupation, un temps de fin d'occupation et un poids (qui peut, par exemple, être lié à l'argent rapporté par cette occupation). Le gestionnaire doit sélectionner un certain nombre n de demandes compatibles entre elles (c'est-à-dire sans chevauchement des durées d'occupation) de façon à maximiser le poids de la somme des demandes satisfaites. Nous

conviendrons de représenter chaque demande par un objet de type défini par

```
type demande = {numero:int; deb:int; fin:int; poids: int};;
```

On suppose donné le tableau `tDemande` des demandes.

Recherche exhaustive

On peut choisir d'essayer toutes les parties de l'ensemble des demandes, et parmi les parties sans chevauchement, déterminer celle qui donne le plus grand poids.

Exercice 6.3

On choisit de représenter une partie A de $\llbracket 0, n-1 \rrbracket$ par un tableau `tA` de 0 et 1, de sorte que $tA(i) = 1$ si $i \in A$ et $tA(i) = 0$ sinon.

1) Écrire une fonction `estCompatible` telle que `estCompatible tDemande partie` renvoie un booléen de valeur `true` si et seulement si la sélection de demandes correspondant à la partie représentée par le tableau `partie` ne présente aucun chevauchement.

2) Écrire une fonction `poidsPartie tDemande partie` qui donne le poids total d'une sélection représentée par le tableau `parties`.

3) En déduire une fonction `meilleureSelection tDemande` qui renvoie une sélection optimale, c'est-à-dire de plus grand poids possible parmi les sélections sans chevauchement.

4) Évaluer la complexité de cet algorithme.

Algorithme glouton dans un cas simple

Le problème est simplifié dans le cas où toutes les demandes ont le même poids.

Exercice 6.4

Cas où toutes les demandes sont de même poids

On se place dans le cas particulier, où toutes les demandes sont de même poids. On veut tenter d'utiliser un algorithme glouton.

1) On pourrait commencer par la demande qui commence le plus tôt, puis choisir celle qui commence le plus tôt parmi les demandes compatibles restantes, etc. Justifier qu'un tel algorithme ne donne pas nécessairement la solution optimale.

2) Montrer que l'on ne peut pas non plus commencer par choisir l'occupation la plus courte.

3) a) Montrer que l'on obtient une solution optimale en choisissant à chaque étape parmi les demandes compatibles restantes celle qui termine le plus tôt.

b) Programmer cette approche.

c) Montrer que l'on peut obtenir une complexité en $O(n \ln(n))$.

6.2 Backtracking

Le retour sur trace cherche à résoudre des problèmes de satisfaction de contraintes (optimisation ou décision), en testant systématiquement l'ensemble des affectations potentielles du problème. Le paradigme du Backtracking consiste à sélectionner une variable du problème, et pour chaque affectation possible de cette variable, à tester récursivement si une solution valide peut être construite à partir de cette affectation partielle. Si aucune solution n'est trouvée, la méthode abandonne et revient sur les affectations qui auraient été faites précédemment.

En fait, le backtracking, consiste à parcourir en profondeur l'arbre de décision du problème: on descend chaque branche autant que possible jusqu'à trouver une solution du problème, ou tomber sur une impasse (impossibilité de poursuivre la construction d'une solution globale), dans le cas d'une impasse on revient en arrière.

6.2.1 Exemple du problème de la partition d'une liste d'entiers

On se donne une liste d'entiers positifs. On cherche à savoir si l'on peut les répartir en deux listes d'entiers de même somme.

Exercice 6.5

Écrire une fonction `peut_partitionner (lst: int list): bool` qui prend en argument une liste d'entiers positifs et retourne `true` s'il est possible de la diviser en deux sous-ensembles de somme égale, et `false` sinon.

Adapter cette fonction pour écrire une fonction `partitionne (lst: int list): (int list * int list) option` qui renvoie `None` ou une option donnant le couple de deux listes représentant cette partition si elle existe.

6.2.2 Coloriage de graphe

On se donne un graphe non orienté de sommets numérotés de 0 à $n - 1$. Ce graphe est représenté par un tableau `g` de liste de voisins: `g.(i)` est la liste des voisins du sommet `i`. On cherche une coloration de chacun des sommets avec k couleurs de sorte que deux sommets voisins n'aient pas la même couleur. Les couleurs sont des entiers numérotés à partir de k . On représente une coloration par le tableau des couleurs `color`, de sorte que `color.(i)` donne la couleur du sommet `i`.

6.2.3 Exemple du problème des Reines

Comment placer n reines sur un échiquier (n, n) de sorte qu'aucune reine ne menace une autre?

6.2.4 Exemple du problème de l'enfilage de perles

On dispose de perles de trois couleurs 0, 1 et 2. On cherche à fabriquer une séquence aussi longue que possible de perles de sorte que l'on ne puisse jamais trouver deux sous-séquences non vides adjacentes identiques.

6.3 Introduction à la programmation dynamique

Exemple: découpage de chaînes de caractères.

Exemple: Justification de textes

Remarques

(–) La première étape dans une résolution par programmation dynamique d'un problème $\mathcal{P}$, est la découverte d'une formule de récursion permettant d'exprimer la résolution de notre problème $\mathcal{P}$ en fonction de petits problèmes d'ordre inférieur. L'efficacité de la résolution par PD, dépend de l'efficacité du choix de ces sous-problèmes et de la formule de récursion.

(–) La deuxième étape consiste à analyser l'ordre dans lequel les sous-problèmes sont résolus, puis à utiliser la structure de mémorisation adaptée.

(–) Quand c'est raisonnablement possible, on cherche la structure de taille minimale contenant toutes les solutions (de sous-problèmes) à mémoriser, on analyse l'ordre dans lequel les problèmes sont résolus, et l'on donne un algorithme impératif.

6.4 Programmation dynamique avec mémorisation

6.4.1 Paradigme

La programmation dynamique avec mémorisation est un principe de programmation qui consiste à résoudre un problème en:

- Se donnant une formule de récursion pour exprimer la résolution du problème posé en se ramenant à la résolution d'un ou plusieurs sous problème plus simple.
- Mettant en place une structure de donnée adaptée au stockage des valeurs correspondant aux solutions des sous-problèmes déjà calculés.
- Résolvant le problème récursivement, en testant à chaque appel récursif, la présence dans notre structure de la solution au sous-problème convoqué:
 - Si la solution au sous-problème convoqué est déjà présente, inutile de recalculer la solution,
 - Si la solution au sous-problème convoqué n'est pas présente, alors on lance effectivement l'appel récursif et l'on stocke cette nouvelle solution dans la structure.

De manière générale, le stockage des solutions peut se faire avec un dictionnaire (table de hachage), mais on peut souvent se contenter d'utiliser une matrice ou un tableau.

La mémoïsation est très étroitement liée à la programmation dynamique, mais ici l'approche est plutôt **du haut vers le bas**, il s'agit d'une approche paresseuse pour mettre en place un algorithme récursif lorsque l'on dispose d'une formule de récursion et que l'on veut éviter de calculer de nombreuses fois la solution à un même problème (qui engendrerait très vite des coûts exponentiels puisque l'algorithme est récursif).

6.4.2 Exemple 1: calcul des termes de la suite de Fibonacci

On peut calculer le terme d'ordre n de la suite de fibonacci en utilisant:

```
let fibo n =
  let t_res = Array.make (n+1) (-1) in
  t_res.(0) <- 0;
  t_res.(1) <- 1;
  let rec fibaux k =
    match k with
    | 0 -> 0
    | 1 -> 1
    | _ when t_res.(k) = (-1) -> (* alors F_k n'est pas encore connu *)
      begin
        t_res.(k) <- fibaux (k-1) + fibaux (k-2);
        t_res.(k);
        (* on le calcule recursivement, et on remplit t_res *)
      end
    | _ -> t_res.(k) (* F_k est deja connu, on le renvoie directement *)
  in fibaux n;
```

Ptitexo 1. Discuter de la complexité spatiale et temporelle de cette fonction.

6.4.3 Exemple 2: Calcul d'une suite récurrente

On se donne une fonction $f : \mathbb{N} \mapsto \mathbb{N}$ et l'on cherche à calculer le terme d'ordre n d'une suite récurrente définie par les relations:

$$u_0 = a \quad \forall n \in \mathbb{N}^*, u_n = u_{\lfloor \frac{n}{2} \rfloor} + u_{\lfloor \frac{2n}{3} \rfloor} + f(n).$$

On peut naïvement procéder de la manière suivante:

```
let rec calcul_naif n f a =
  match n with
  | 0 -> a
  | _ -> (calcul_naif (n/2) f a) +
    (calcul_naif ((2*n)/3) f a) + f(n);;
```

Ptitexo 2. Dessiner l'arbre des appels récursifs pour $n = 29$, $n = 44$, $n = 66$, $n = 100$. Commentaires?

Ptitexo 3. Proposez une fonction utilisant de la mémoïsation pour améliorer à peu de frais cette fonction. Dessiner l'arbre des appels récursifs pour cette nouvelle fonction.

Au vu de l'arbre des appels récursifs, on comprend que l'on ne va utiliser qu'une petite partie du tableau des entiers de 0 à n pour stocker les valeurs de u_n au fur et à mesure de leur calcul.

Il est donc tout à fait indiqué d'utiliser un dictionnaire pour stocker ces valeurs, on va utiliser un dictionnaire dont les clés seront les $n \in \mathbb{N}$ et les valeurs de u_n .

Cela donne:

```
let calcul_table n f a=
  (* creation d'une table pour stocker les u_n qu'on a calculees*)
  (* les clefs seront les n, les valeurs les u_n*)
  let table = Hashtbl.create n
  (* * ajoutons de suite la valeur connue:u_0*)
  in
  Hashtbl.add table 0 a;
  let rec aux k =
    match k with
    0-> a
  | _ when not (Hashtbl.mem table k) -> (* u_k pas encore connu*)
      let v = aux (k/2) + aux (2*k/3) + f(k) in
      Hashtbl.add table k v; (* on le calcule, le met dans la table*)
      v; (* on renvoie u_k*)
  | _-> Hashtbl.find table k (* u_k connu, on le renvoie*)
  in
  aux n;;
```

6.5 Programmation dynamique impérative

6.5.1 Paradigme

La programmation dynamique est une méthode de programmation souvent utilisée (notamment) pour résoudre des problèmes d'optimisation. Le paradigme est le suivant:

- On détermine une suite de sous-problèmes dont la résolution permettra de résoudre le problème posé.
- On résout chaque sous-problème du plus petit au plus grand.
- On en déduit la solution optimale au problème posé, cela nécessite parfois de remonter les calculs de l'étape précédente.

La programmation dynamique impérative est donc plutôt une méthode de bas en haut.

6.5.2 Exemple 1: Plus grand carré de 1 dans une matrice

Énoncé du problème

Soit $M \in M_{n,p}(\{0,1\})$ une matrice à coefficients dans $\{0,1\}$. On cherche à déterminer la taille du plus grand carré extrait de cette matrice et ne contenant que des 1. On ne s'autorisera qu'à extraire des carrés formés de cases contigües.

Résolution par programmation dynamique

Identification des sous-problèmes à résoudre

Pour tout couple $(i,j) \in \llbracket 1,n \rrbracket \times \llbracket 1,p \rrbracket$, on peut calculer $T_{i,j}$, la taille du plus grand carré composé uniquement de 1 et dont le coin en bas à droite est d'indice (i,j) .

Résolution des sous-problèmes

On va déterminer la matrice $T = (T_{i,j})_{0 \leq i \leq n, 0 \leq j \leq p} \in M_{n,p}(\mathbb{R})$.

Ptitexo 4.

1. Que vaut $T_{i,j}$ si $i = 0$ ou $j = 0$?
2. Que vaut $T_{i,j}$ si $M_{i,j} = 0$?
3. Si $M_{i,j} \neq 0$ et $i \in \llbracket 2, n \rrbracket$ et $j \in \llbracket 2, p \rrbracket$, exprimer $T_{i,j}$ en fonction de $T_{i-1,j}$, $T_{i-1,j-1}$ et $T_{i,j-1}$?

Résolution du problème initial

La taille du plus grand carré composé uniquement de 1 à l'intérieur de M est donc le maximum des coefficients de T .

Ptitexo 5. En déduire une fonction **plusGrandCarre** M qui renvoie la taille du plus grand carré ne contenant que des 1 contenu dans M . Vérifier que l'on obtient ainsi une fonction de complexité $O(np)$.

6.5.3 Exemple 1 bis: Plus grand rectangle composé de 1

Soit $M \in M_{n,p}(\{0, 1\})$ une matrice à coefficients dans $\{0, 1\}$. On cherche à déterminer la taille du plus grand rectangle de cette matrice et ne contenant que des 1. On ne s'autorisera qu'à extraire des rectangles formés de cases contiguës. (On peut trouver une solution linéaire en le nombre de cases de la matrice).

6.5.4 Exemple 2: Problème du rendu de monnaie

On reprend le problème formulé dans la section précédente, pour le résoudre par programmation dynamique:

Identification des sous-problèmes à résoudre

Pour toute somme i inférieure à v , on cherche le nombre de pièces minimal nécessaires pour obtenir la somme v . On notera $res(i)$ ce nombre de pièces minimal.

Résolution des sous-problèmes

Ptitexo 6. Pour quelles valeurs de i connaît-on immédiatement la valeur de $res(i)$? Soit $i \in \llbracket 2, v \rrbracket$. Comment obtenir le nombre $res(i)$ en fonction des $res(j)$ pour $j \in \llbracket 1, i-1 \rrbracket$? En déduire une fonction **rendu** v s qui renvoie le tableau des $res(i)$.

Résolution du problème initial

Le tableau res étant connu, il suffit de lire $res(v)$ pour connaître le nombre de pièces minimal à utiliser pour former la somme v .

Ptitexo 7. Modifier le programme précédent pour fabriquer la fonction **rendu2** v s qui renvoie le tableau t tel que $t.(i)$ est la valeur obtenue en ôtant à i la première pièce nécessaire à la fabrication de la solution optimale de somme i .
En déduire **rendre_monnaie** v s qui renvoie la liste des pièces à rendre dans la solution optimale pour v .

6.5.5 Exemple 3: Chemin de poids minimal dans une matrice**Énoncé du problème**

On se donne une matrice $M = (m_{i,j})_{(i,j) \in \llbracket 0, n-1 \rrbracket \times \llbracket 0, p-1 \rrbracket} \in M_{n,p}(\mathbb{N})$, on se déplace dans la matrice en partant du coefficient $(0,0)$ en allant d'une unité vers la droite ou vers le bas à chaque déplacement. On appelle poids du chemin ainsi obtenu, la somme des coefficients de la matrice rencontrés sur ce chemin. On cherche à déterminer le poids minimal d'un chemin joignant $(0,0)$ à $(n-1, p-1)$.

Résolution par programmation dynamique

Cherchons par exemple, le poids minimal d'un chemin reliant la case $(0, 0)$ à la case (i, j) pour n'importe quel couple (i, j) d'indices valides?

Exercice 6.6

- 1) En utilisant de la programmation dynamique écrire une fonction qui calcule le poids minimal d'un chemin joignant $(0, 0)$ à $(n - 1, p - 1)$.
 - 2) Écrire une fonction qui calcule et renvoie un chemin de poids minimal.
-

6.5.6 Exemple 4: Problème d'allocations de salles

On se place dans le cas général, où toutes les demandes sont affectées d'un certain poids, et l'on cherche à maximiser le poids total de la liste des demandes retenues.

Première approche récursive

On commence par réordonner le tableau des demandes par date de fin croissante. On obtient une suite de demandes numérotées de 0 à $n - 1$. Dès lors, une solution optimale est de l'une des deux formes suivantes:

- On a une solution optimale qui contient la dernière demande (celle qui finit le plus tard). Dans ce cas, si l'on note $r(n - 1)$ le numéro de la dernière demande compatible avec la demande $n - 1$, alors une solution optimale contient:
 - > les mêmes demandes que celles d'une solution optimale en ne considérant que les demandes de numéro $\leq r(n - 1)$,
 - > et la demande $r(n - 1)$.
- On a une solution optimale qui ne contient pas la dernière demande, en ce cas, elle contient les mêmes demandes que celles de la solution optimale en ne considérant que les $n - 1$ premières demandes.

Autrement dit, si l'on note $res(j)$ le poids maximal que l'on peut obtenir en ne considérant que les demandes d'indice $\leq j$, on a:

$$res(n - 1) = \max \{ res(n - 2); (\text{poids de la demande } n - 1) + res(r(n - 1)) \}$$

Exercice 6.7

- 1) On note $r(k)$ le numéro de la dernière demande compatible avec la demande de numéro k . Évaluer la complexité du calcul du tableau des $r(k)$ pour tout $k \in \llbracket 0, n - 1 \rrbracket$.
 - 2) Utiliser pour la formule précédente pour construire un algorithme récursif naïf donnant le poids d'une solution optimale.
 - 3) Évaluer la complexité de cet algorithme. On pourra commencer par dessiner l'arbre des appels récursifs lorsque les demandes sont (au format deb, fin, poids): $(2; 8; 2)$, $(10; 14; 1)$, $(4; 7; 12)$, $(6; 10; 4)$, $(10; 14; 2)$ et $(0; 6; 4)$.
-

Résolution par programmation dynamique

On va maintenant utiliser la programmation dynamique pour mettre en place une solution efficace.

Exercice 6.8

Mise en place d'une programmation dynamique

On observe qu'il suffit en fait de calculer $res(0); \dots; res(n - 1)$, et que le calcul de $res(i)$ se fait simplement si l'on connaît $res(0); \dots; res(i - 1)$ et le tableau $r(0); \dots; r(n - 1)$. Ce qui conduit naturellement à notre programmation dynamique.

1) Écrire la fonction `tableauR tabDemandeTrie` qui prend en argument un tableau de demande triées par date de fin croissante, et renvoie le tableau $[r(0); \dots; r(n-1)]$.

2) En utilisant, de la programmation dynamique écrire la fonction `valAllocMax` qui prend en argument argument le tableau des demandes et renvoie le poids d'une solution optimale.

3) Adapter la fonction précédente, pour qu'elle renvoie la liste des numéros de demandes d'une solution optimale.

6.5.7 Alignement de séquences

On considère un alphabet $\mathcal{A} = \{A, T, G, C\}$ et un caractère Δ ne faisant pas partie de cet alphabet.

Lorsque u et v sont deux mots sur l'alphabet $\mathcal{A}$, on appelle alignement des mots u et v , toute matrice M à coefficients dans $\mathcal{A} \cup \{\Delta\}$ de deux lignes et p colonnes vérifiant:

(-) La première ligne de M contient les caractères de u dans l'ordre éventuellement entrecoupés du caractère Δ .

(-) La seconde ligne de M contient les caractères de v dans l'ordre éventuellement entrecoupés du caractère Δ .

(-) Aucune colonne de M ne vaut $\begin{pmatrix} \Delta \\ \Delta \end{pmatrix}$.

Par exemple, lorsque $u = AGTTGC$ et $v = TAGTC$, la matrice suivante est un alignement de u et v :

$$v : \begin{pmatrix} A & \Delta & G & T & T & \Delta & G & C \\ T & A & \Delta & G & \Delta & T & \Delta & C \end{pmatrix}$$

On appelle fonction de score toute application f de $(\mathcal{A} \cup \{\Delta\})^2$ vers $\mathbb{R}$. Le score $S(M)$ d'un alignement $M \in M_{2,n}(\mathcal{A} \cup \{\Delta\})$ est alors

$$S(M) = \sum_{k=1}^n f(M_{1,k}, M_{2,k}).$$

Par exemple, on peut définir une fonction de score par:

$$f(a, b) = 4 \text{ si } a = b \in \mathcal{A}$$

$$f(a, b) = -1 \text{ si } a \in \mathcal{A} \text{ et } b = \Delta$$

$$f(a, b) = -1 \text{ si } a = \Delta \text{ et } b \in \mathcal{A}.$$

$$f(a, b) = 0 \text{ si } a \in \mathcal{A} \text{ et } b \in \mathcal{A} \text{ et } a \neq b.$$

On cherche, les mots u et v étant fixés, un alignement de score maximal.

Résolution par programmation dynamique

Notons $u = u_1 \dots u_m$ et $v = v_1 \dots v_n$ nos deux mots sur l'alphabet $\mathcal{A}$. Pour tout $(i, j) \in \llbracket 0, m \rrbracket \times \llbracket 0, n \rrbracket$, on note $s_{i,j}$ le meilleur score d'alignement possible entre les mots $u_1 \dots u_i$ et $v_1, \dots, v_j$. On va fabriquer la matrice $(s_{i,j})_{0 \leq i \leq m, 0 \leq j \leq n}$.

Exercice 6.9

- 1) a) Remarquer que $s_{0,0} = 0$ et pour tout $j \in \llbracket 1, n \rrbracket$, $s_{0,j} = s_{0,j-1} + f(\Delta, v_j)$ et pour tout $i \in \llbracket 1, m \rrbracket$, $s_{i,0} = s_{i-1,0} + f(u_i, \Delta)$.
 b) Remarquer que si $(i, j) \in \llbracket 1, n \rrbracket \times \llbracket 1, m \rrbracket$:

$$s_{i,j} = \max \{s_{i-1,j-1} + f(u_i, v_j), s_{i,j-1} + f(\Delta, v_j), s_{i-1,j} + f(u_i, \Delta)\}.$$

- 2) En déduire une fonction `bestscore u v f` qui calcule le meilleur score pour un alignement des mots u et v , lorsque l'on passe en argument les mots et la fonction de score.
 3) (*) Comment adapter cette fonction à la détermination d'un meilleur alignement?

6.5.8 Autres exemples de PD

Ptitexo 8. Dans un tableau d'entiers $X = [x_0, \dots, x_{n-1}]$, on appelle sous-suite croissante, une suite $x_{i_1}, x_{i_2}, \dots, x_{i_k}$ tels que $x_{i_1} \leq x_{i_2} \leq \dots \leq x_{i_k}$ et $0 \leq i_1 < i_2 < \dots < i_k \leq n-1$. On note $\text{res}(i)$ la longueur de la plus grande sous-suite croissante se finissant par x_i .

1. Montrer que pour tout $i \in \llbracket 1, n-1 \rrbracket$, on a:

$$\text{res}(i) = 1 + \max \left\{ \text{res}(j) \mid j < i \text{ et } x_j \leq x_i \right\}.$$

2. En déduire une fonction de calcul du tableau $[\text{res}(0), \dots, \text{res}(n-1)]$.
 3. En déduire une fonction de calcul de la longueur de la plus grande sous-suite croissante se finissant par x_i .
 4. Adapter pour écrire une fonction qui renvoie une sous-suite croissante de longueur maximale.

Ptitexo 9. On considère une liste $x_1, \dots, x_n$ d'entiers strictement positifs et un entier $M > 0$. On note pour tout $(i, j) \in \llbracket 0, n \rrbracket \times \llbracket 0, M \rrbracket$, $b(i, j)$ le booléen qui vaut `true` s'il existe une partie de $\{x_1, \dots, x_i\}$ de somme égale à j et `false` sinon.

1. Donner une relation de récurrence sur les coefficients de la matrice $(b_{i,j})_{0 \leq i \leq n, 0 \leq j \leq M}$.
 2. En utilisant la programmation dynamique, en déduire une fonction qui nous dit s'il existe une partie de $\{x_1, \dots, x_n\}$ de somme égale à M .

Ptitexo 10. On se donne une partie E de $\mathbb{N}^*$. En s'inspirant de la méthode de l'exercice précédent, écrire une fonction qui calcule, si elle existe, la plus grande somme inférieure ou égale à M que l'on peut obtenir en sommant les éléments d'une partie de E .

Exercice 6.10**Multiplication matricielle**

On se donne des matrices $A_1, \dots, A_n$ d'entiers. On note pour tout $i \in \llbracket 1, n \rrbracket$, c_i le nombre de colonnes de A_i et ℓ_i le nombre de lignes de A_i . On suppose que ces matrices ont des dimensions compatibles, au sens où le produit $A_1 \dots A_n$ est bien défini.

- 1) Montrer, à l'aide d'exemples bien choisis, que bien que le produit matriciel soit associatif au sens mathématique, le nombre de multiplications d'entiers effectuées pour le calcul de $A_1 \times \dots \times A_n$ dépend de l'ordre dans lequel les produits sont effectués.

- 2) On note pour $1 \leq i, j \leq n$ le nombre minimal de multiplications d'entiers nécessaires pour effectuer le produit $A_i \times \dots \times A_j$. Déterminer une formule de récurrence sur la matrice $C = (C_{i,j})_{1 \leq i, j \leq n}$.

3) En déduire une fonction de calcul du minimum de multiplications d'entiers à effectuer pour calculer le produit matriciel $A_1 \dots A_n$.

4) (*) Écrire une fonction qui renvoie une chaîne de caractères donnant le parenthésage d'un ordre de calcul optimal. Par exemple, on pourra renvoyer la chaîne $(1 * ((2 * 3) * 4)) * 5$.

Ptitexo 11. Parenthésage. On se donne une expression arithmétique commençant par un entier et finissant par un entier, deux entiers successifs sont séparés par le symbole $+$ ou le symbole $-$. Par exemple, on peut considérer l'expression: $e = 14 - 5 + 6 - 9 - 1 - 8 + 5 - 6$. Le but du jeu est ici, d'ajouter des parenthèses aux bons endroits pour obtenir la somme maximale. Par exemple, avec l'expression $1 - 3 - 2 - 5$, on peut obtenir: $1 - 3 - 2 - 5 = -9$ ou $1 - (3 - 2) - 5 = -5$ ou $(1 - 3) - (2 - 5) = 1$ ou $1 - (3 - 2 - 5) = 5$ ou $1 - (3 - (2 - 5)) = -5$. On considère que l'on a le droit d'ajouter des parenthèses uniquement devant un nombre, on ne peut pas juxtaposer des parenthèses faisant apparaître l'écriture implicite d'un produit.

L'expression donnée en entrée du problème est codée par une liste dont chaque case est un entier ou un objet à choisir parmi '+' ou '-'. Par exemple, l'expression e est codée par le tableau $e = [14, '-', 5, '+', 6, '-', 9, '-', 1, '-', 8, '+', 5, '-', 6]$.

Écrire de façon efficace une fonction qui prend en argument une liste représentant cette expression et renvoie la somme maximale que l'on peut obtenir en adaptant le parenthésage.

Exercice 6.11
Problème de gros sac

On dispose d'un sac à dos qui craque si son chargement dépasse le poids W . On dispose d'objets numérotés de 1 à n de valeur $v_1, \dots, v_n$ et de poids respectifs $w_1, \dots, w_n$.

1) Écrire une fonction qui renvoie la valeur maximale d'un chargement qui puisse tenir dans le sac à dos. *Indication:* On notera $res(w, i)$ la valeur maximale d'un chargement admissible par le sac à dos construit en chargeant des objets choisis par les objets 1 à n .

2) Adapter cette fonction pour qu'elle nous renvoie un chargement de valeur maximale.

6.6 Diviser pour Régner

6.6.1 Paradigme

La méthode dite du diviser pour régner est un paradigme de programmation qui a pour but de réduire drastiquement la complexité des algorithmes en procédant de la manière suivante:

- On **divise** le problème à résoudre en plusieurs petits problèmes (généralement de taille à peu près égale),
- On résout (récursivement) chaque sous problème,
- On combine ensuite les solutions trouvées à chaque sous-problème pour résoudre le problème posé. (C'est la phase régner).

La pertinence d'une telle approche repose sur la possibilité de trouver un moyen d'effectuer astucieusement et rapidement la phase REGNER.

6.6.2 Exemple1: les tris

Nous avons déjà observé que les tris sur les listes et le quicksort sont des algorithmes de tri efficaces basés sur un principe de type diviser pour régner.

6.6.3 Exemple 2: Exponentiation rapide

6.6.4 Exemple 3: Calcul du nombre d'inversions

Ptitexo 12. Déterminer le nombre d'inversions dans un tableau t de n entiers deux à deux distincts. (On rappelle qu'une inversion est un couple de deux indices i et j tels que $i < j$ et $t.(i) > t.(j)$.)

6.6.5 Exemple 4: Multiplication par algorithme de Karatsuba

Énoncé du problème: On cherche à effectuer efficacement le produit de deux polynômes connaissant la liste de leurs coefficients.

Résolution: On se restreint au cas où les deux polynômes P et P_1 sont de degré inférieur à $2k - 1$.

- **Diviser:** Il existe des polynômes A_1, A_2, B_1 et B_2 de degré $\leq k - 1$ chacun tels que $P_1 = A_1X^k + B_1$ et $P_2 = A_2X^k + B_2$.
- On peut calculer récursivement les produits A_1A_2, B_1B_2 et $(A_1 - B_1)(A_2 - B_2)$.
- **Régner:** On obtient alors P_1P_2 par la formule

$$P_1P_2 = A_1A_2X^{2k} + (A_1A_2 + B_1B_2 - (A_1 - B_1)(A_2 - B_2))X^k + B_1B_2.$$

Exercice 6.12

Algorithme de Karatsuba

Énoncé du problème: On cherche à effectuer efficacement le produit de deux polynômes P_1 et Q_1 connaissant la liste de leurs coefficients.

1) On suppose, dans cette question seulement, que les coefficients des deux polynômes P_1 et Q_1 sont stockés dans deux tableaux. Détailler en langage naturel, l'algorithme banal utilisable pour obtenir les coefficients du produit P_1Q_1 . Évaluer la complexité de cet algorithme en nombre de multiplications entre les coefficients.

Dans toute la suite, pour simplifier notre étude, on se restreint au cas où l'on multiplie deux polynômes de degré inférieur à $2^p - 1$ (avec $p \in \mathbb{N}^*$), c'est-à-dire deux polynômes que l'on peut représenter par une liste de coefficients de longueur 2^p .

2) Justifier que l'on ne perd pas de généralité en se restreignant au cas du produit de deux polynômes de degré $\leq 2^p - 1$. Comment se ramener à ce cas, à partir de deux polynômes de degrés quelconque?

Notons $k = 2^{p-1}$, il existe donc des polynômes A_1, B_1, A_2 et B_2 de degré inférieur ou égal à $k - 1$ tels que

$$P_1 = A_1X^k + B_1 \quad P_2 = A_2X^k + B_2.$$

3) **Première tentative:** On peut avoir l'idée de l'algorithme récursif suivant:

- Calculer récursivement les produits A_1A_2, A_1B_2, A_2B_1 et B_1B_2
- Reconstituer P_1P_2 en utilisant:

$$P_1P_2 = A_1A_2X^{2k} + (A_1B_2 + A_2B_1)X^k + B_1B_2.$$

a) On note N_p , le nombre de multiplications entre coefficients utilisées pour faire le produit de deux polynômes de degré inférieur à $2^p - 1$ par cet algorithme. Donner une formule de récurrence sur la suite (N_p) .

b) En déduire (N_p) en fonction de p . Évaluer la complexité de cet algorithme et commenter.

Algorithme de Karatsuba Avec les notations précédentes l'algorithme de Karatsuba est basé sur l'observation suivante:

$$P_1P_2 = (A_1A_2)X^{2k} + (A_1A_2 + B_1B_2 - (A_1 - B_1)(A_2 - B_2))X^k + B_1B_2.$$

Il n'y a donc besoin de calculer récursivement que trois produits de polynômes de degré $\leq 2^{p-1} - 1$: les produits A_1A_2, B_1B_2 et $(A_1 - B_1)(A_2 - B_2)$.

4) En utilisant la méthode de la question 3, évaluer en fonction de $\deg(P_1)$ et $\deg(P_2)$ la complexité de cet algorithme en nombre de multiplications. En conclure qu'il est intéressant de mettre en place cet algorithme et donc d'aborder les questions qui suivent.

Dans toute la suite, un polynôme est représenté par une liste de coefficients, en tête de liste, on met les coefficients correspondant aux plus grandes puissances. Par exemple, le polynôme $Pex = 7X^3 + 2X^2 + X + 1$ est représenté par les listes $[7; 2; 1; 1]$ ou $[0; 0; 7; 2; 1; 1]$.

5) Mise à une taille puissance de 2

a) Écrire une fonction `lg liste` qui renvoie la longueur de la liste `liste`. Cette fonction sera récursive terminale ou fera appel à une fonction récursive terminale.

b) Écrire une fonction `ajoute_zero liste nb` qui ajoute `nb` zéros en tête de la liste `liste`. Cette fonction sera récursive terminale ou fera appel à une fonction récursive terminale.

c) Écrire une fonction `taille n` qui prend en argument un entier n et renvoie la valeur du plus petit entier p tel que $n \leq 2^p$.

d) Écrire une fonction `expo_rapide k` basée sur l'algorithme d'exponentiation rapide qui renvoie la valeur de 2^k .

e) Écrire une fonction `agrandir poly` qui prend en argument la liste `poly` des coefficients du polynôme P et renvoie une liste de taille 2^p représentant le même polynôme, on cherchera évidemment à minimiser p . Par exemple, pour `pex = [1; 2; 3; 4; 5]`, l'appel de `agrandir pex` renverra `[0; 0; 0; 1; 2; 3; 4; 5]`.

6) diviser

Écrire une fonction `diviser p`, qui prend en argument une liste `p` de taille n et renvoie un couple de deux listes `l1` et `l2`, la première liste contenant les $\lfloor \frac{n}{2} \rfloor$ premiers éléments de `l` et la deuxième contenant la deuxième partie de `l`.

7) Ajouter, soustraire

a) Écrire une fonction `somme p1 p2` qui renvoie la liste représentant la somme des polynômes P_1 et P_2 représentés par les listes `p1` et `p2`. On suppose que `p1` et `p2` sont de même longueur, sinon on renvoie une erreur.

b) Écrire une fonction `soustraire p1 p2` qui renvoie la liste représentant la différence $P_1 - P_2$ des polynômes P_1 et P_2 représentés par les listes `p1` et `p2`.

8) Multiplication d'après Karatsuba

a) Écrire `karatsuba p1 p2` qui calcule le produit de deux polynômes représentés par une liste de même longueur, cette longueur étant supposée être une puissance de 2. tester.

b) En déduire `multiK p1 p2` qui effectue le produit de deux polynômes de taille quelconque.

6.6.6 Autres exemples:

Calcul de la plus courte distance entre deux points,

Chasse aux fantômes (Voir sujet d'entraînement tiré de Centrale 2009, par ailleurs),

Calcul de ligne d'horizon,

calcul du nombre d'inversions dans un tableau d'entiers,

recherche d'un élément majoritaire.